# 22. 테스트 프레임워크 및 시뮬레이션 분석

## 개요

Lighthouse는 포괄적인 테스트 프레임워크와 시뮬레이션 도구들을 제공하여 Ethereum 2.0 합의 클라이언트의 정확성과 안정성을 보장합니다. 이 문서는 `testing/` 디렉토리의 다양한 테스트 도구들과 로컬 테스트넷 구성 방법을 상세히 분석합니다.

## 테스트 프레임워크 구조

### 1. 전체 테스트 아키텍처

```
testing/
├── ef_tests/ # Ethereum Foundation 공식 테스트 벡터
├── execution_engine_integration/ # 실행 엔진 통합 테스트
├── network_testing/ # 네트워크 테스트 도구
├── node_test_rig/ # 노드 테스트 리그 (핵심 테스트 인프라)
├── simulator/ # 전체 네트워크 시뮬레이션
├── state_transition_vectors/ # 상태 전이 테스트 벡터
├── validator_test_rig/ # 검증자 테스트 리그
└── web3signer_tests/ # Web3Signer 통합 테스트
```

### 2. 핵심 테스트 컴포넌트

### Node Test Rig ( `node_test_rig/` )

Node Test Rig는 Lighthouse의 핵심 테스트 인프라로, 프로덕션과 동일한 타입을 사용하여 beacon node와 validator client를 인프로세스로 실행할 수 있게 해줍니다.

**주요 구조체들:**

```
// testing/node_test_rig/src/lib.rs
pub struct LocalBeaconNode<E: EthSpec> {
pub client: ProductionClient<E>,
pub datadir: TempDir,
}

pub struct LocalValidatorClient<E: EthSpec> {
pub client: ProductionValidatorClient<E>,
pub files: ValidatorFiles,
}

pub struct LocalExecutionNode<E: EthSpec> {
pub server: MockServer<E>,
pub datadir: TempDir,
}
```

**핵심 기능:**

- **프로덕션 환경 시뮬레이션**: 실제 프로덕션 코드와 동일한 타입 사용
- **임시 디렉토리 관리**: 테스트용 데이터 디렉토리 자동 생성 및 정리

- **HTTP API 접근**: 외부 클라이언트처럼 노드에 접근할 수 있는 인터페이스 제공

```
impl<E: EthSpec> LocalBeaconNode<E> {
/// 프로덕션 beacon node를 tokio 런타임에서 시작
pub  async  fn  production(
context: RuntimeContext<E>,
mut  client_config: ClientConfig,
) -> Result<Self, String> {
// 임시 디렉토리 생성
let  datadir = TempBuilder::new()
.prefix("lighthouse_node_test_rig")
.tempdir()
.expect("should create temp directory for client datadir");


client_config.set_data_dir(datadir.path().into());
// 타임아웃과 함께 beacon node 시작
timeout(
STARTUP_TIMEOUT,
ProductionBeaconNode::new(context, client_config),
)
.await
.map_err(|_| format!("Beacon node startup timed out after {:?}", STARTUP_TIMEOUT
.map(move |client| Self {
```

```
client: client.into_inner(),
datadir,
})
}

/// 외부 접근을 위한 HTTP 클라이언트 반환
pub fn remote_node(&self) -> Result<BeaconNodeHttpClient, String> {
let listen_addr = self
.client
.http_api_listen_addr()
.ok_or("A remote beacon node must have a http server")?;

let beacon_node_url: SensitiveUrl = SensitiveUrl::parse(
format!("http://{}:{}", listen_addr.ip(), listen_addr.port()).as_str(),
)
.map_err(|e| format!("Unable to parse beacon node URL: {:?}", e))?;
Ok(BeaconNodeHttpClient::from_components(
beacon_node_url,
beacon_node_http_client,
Timeouts::set_all(HTTP_TIMEOUT),
))
}
```

```
}
```

### 테스트 설정 함수들

```
pub fn testing_client_config() -> ClientConfig {
let mut client_config = ClientConfig::default();

// OS가 사용 가능한 포트를 선택하도록 0으로 설정
client_config
.network
.set_ipv4_listening_address(std::net::Ipv4Addr::UNSPECIFIED, 0, 0, 0);
client_config.network.upnp_enabled = false;
client_config.http_api.enabled = true;
client_config.http_api.listen_port = 0;

// Interop 제네시스 설정
client_config.genesis = ClientGenesis::Interop {
validator_count: 8,
genesis_time: now,
};

// 시뮬레이터 테스트를 위한 히스토릭 상태 재구성 활성화
client_config.chain.reconstruct_historic_states = true;
// 안정적인 HTTP 응답을 위한 beacon processor 워커 수 설정
```

```
client_config.beacon_processor.max_workers = 4;


client_config

}
```

# 시뮬레이션 프레임워크

## 1. 네트워크 시뮬레이터 ( `simulator/` )

시뮬레이터는 전체 Ethereum 2.0 네트워크를 로컬에서 시뮬레이션할 수 있는 도구입니다.

### LocalNetwork 구조체

```
// testing/simulator/src/local_network.rs

pub struct LocalNetwork<E: EthSpec> {

inner: Arc<Inner<E>>,

}


pub struct Inner<E: EthSpec> {

pub context: RuntimeContext<E>,

pub beacon_nodes: RwLock<Vec<LocalBeaconNode<E>>>,

pub proposer_nodes: RwLock<Vec<LocalBeaconNode<E>>>,

pub validator_clients: RwLock<Vec<LocalValidatorClient<E>>>,

pub execution_nodes: RwLock<Vec<LocalExecutionNode<E>>>,
```

```
}
```

### 네트워크 생성 및 관리

```
impl<E: EthSpec> LocalNetwork<E> {
/// 로컬 네트워크 생성
pub  async  fn  create_local_network(
client_config: Option<ClientConfig>,
mock_execution_config: Option<MockExecutionConfig>,
network_params: LocalNetworkParams,
context: RuntimeContext<E>,
) -> Result<(LocalNetwork<E>, ClientConfig, MockExecutionConfig), String> {
// 제네시스 시간 계산
let  genesis_time: u64 = (SystemTime::now()
.duration_since(UNIX_EPOCH)
.map_err(|_| "should get system time")?
+ Duration::from_secs(network_params.genesis_delay))
.as_secs();


// 기본 설정 또는 사용자 제공 설정 사용
let  beacon_config = if  let Some(config) = client_config {
config
} else {
default_client_config(network_params, genesis_time)
```

```
};

let execution_config = if let Some(config) = mock_execution_config {
config
} else {
default_mock_execution_config::<E>(&context.eth2_config().spec, genesis_time)
};

let network = Self {
inner: Arc::new(Inner {
context,
beacon_nodes: RwLock::new(vec![]),
proposer_nodes: RwLock::new(vec![]),
execution_nodes: RwLock::new(vec![]),
validator_clients: RwLock::new(vec![]),
}),
};

Ok((network, beacon_config, execution_config))
}

/// 네트워크에 beacon node 추가
pub async fn add_beacon_node(
```

```
&self,
mut beacon_config: ClientConfig,
mock_execution_config: MockExecutionConfig,
is_proposer: bool,
) -> Result<(), String> {
// 첫 번째 노드가 부트노드 역할
let first_bn_exists: bool;
{
let read_lock = self.beacon_nodes.read();
let boot_node = read_lock.first();
first_bn_exists = boot_node.is_some();


if let Some(boot_node) = boot_node {
// 부트노드 ENR을 설정에 추가
beacon_config.network.boot_nodes_enr.push(
boot_node
.client
.enr()
.expect("Bootnode must have a network."),
);
}
}
let (beacon_node, execution_node) = if first_bn_exists {
```

```
self.construct_beacon_node(beacon_config, mock_execution_config, is_proposer)
.await?
} else {
self.construct_boot_node(beacon_config, mock_execution_config)
.await?
};
// 네트워크에 노드 추가
self.execution_nodes.write().push(execution_node);
if  is_proposer {
self.proposer_nodes.write().push(beacon_node);
} else {
self.beacon_nodes.write().push(beacon_node);
}
Ok(())
}
}
```

## 2. 기본 시뮬레이션 (`basic_sim.rs`)

기본 시뮬레이션은 다중 노드 네트워크를 생성하고 다양한 검증을 수행합니다.

### 시뮬레이션 매개변수

```
// testing/simulator/src/basic_sim.rs
```

```
const END_EPOCH: u64 = 16;

const GENESIS_DELAY: u64 = 32;

const ALTAIR_FORK_EPOCH: u64 = 0;

const BELLATRIX_FORK_EPOCH: u64 = 0;

const CAPELLA_FORK_EPOCH: u64 = 0;

const DENEB_FORK_EPOCH: u64 = 0;

const ELECTRA_FORK_EPOCH: u64 = 2;



pub  fn  run_basic_sim(matches: &ArgMatches) -> Result<(), String> {

// CLI 인수 파싱

let  node_count = subcommand_matches

.get_one::<String>("nodes")

.expect("missing nodes default")

.parse::<usize>()

.expect("missing nodes default");

let  validators_per_node = subcommand_matches

.get_one::<String>("validators-per-node")

.expect("missing validators-per-node default")

.parse::<usize>()

.expect("missing validators-per-node default");

let  speed_up_factor = subcommand_matches

.get_one::<String>("speed-up-factor")

.expect("missing speed-up-factor default")
```

```
.parse::<u64>()
.expect("missing speed-up-factor default");
}
```

### 검증자 키스토어 생성

```
// 병렬로 검증자 파일 생성
let validator_files = (0..node_count)
.into_par_iter()
.map(|i| {
println!(
"Generating keystores for validator {} of {}",
i + 1,
node_count
);

let indices =
(i * validators_per_node..(i + 1) * validators_per_node).collect::<Vec<_>>();
ValidatorFiles::with_keystores(&indices).unwrap()
})
.collect::<Vec<_>>();
```

### 네트워크 검증 테스트

시뮬레이션은 다양한 측면에서 네트워크 동작을 검증합니다:

```
let (
finalization,
block_prod,
validator_count,
onboarding,
fork,
sync_aggregate,
transition,
light_client_update,
blobs,
start_node_with_delay,
sync,
) = futures::join!(
// 첫 번째 파이널라이제이션 검증
checks::verify_first_finalization(network.clone(), slot_duration),
// 모든 슬롯에서 블록 생성 검증
checks::verify_full_block_production_up_to(
network.clone(),
Epoch::new(END_EPOCH).start_slot(slots_per_epoch),
slot_duration,
),
// 초기 검증자 수 검증
```

```
checks::verify_initial_validator_count(
network.clone(),
slot_duration,
initial_validator_count,
),
// 검증자 온보딩 검증
checks::verify_validator_onboarding(
network.clone(),
slot_duration,
total_validator_count,
),
// 포크 버전 전환 검증
checks::verify_fork_version(
network.clone(),
Epoch::new(latest_fork_start_epoch),
slot_duration,
latest_fork_version,
),
// 동기화 집계 검증
checks::verify_full_sync_aggregates_up_to(
network.clone(),
Epoch::new(ALTAIR_FORK_EPOCH + 1).start_slot(slots_per_epoch),
Epoch::new(END_EPOCH).start_slot(slots_per_epoch),
```

```
slot_duration,
),
// 전환 블록 파이널라이제이션 검증
checks::verify_transition_block_finalized(
network.clone(),
Epoch::new(TERMINAL_BLOCK / slots_per_epoch),
slot_duration,
true,
),
// 라이트 클라이언트 업데이트 검증
checks::verify_light_client_updates(
network.clone(),
Epoch::new(ALTAIR_FORK_EPOCH).start_slot(slots_per_epoch) + 1,
Epoch::new(END_EPOCH).start_slot(slots_per_epoch),
slot_duration
),
// 블롭 생성 검증 (Deneb 포크 이후)
checks::verify_full_blob_production_up_to(
network.clone(),
Epoch::new(DENEB_FORK_EPOCH).start_slot(slots_per_epoch),
Epoch::new(END_EPOCH).start_slot(slots_per_epoch),
slot_duration
),
```

```
// 지연된 노드 추가
network_1.add_beacon_node_with_delay(
beacon_config.clone(),
mock_execution_config.clone(),
END_EPOCH - 1,
slot_duration,
slots_per_epoch
),
// 노드 동기화 검증
checks::ensure_node_synced_up_to_slot(
network.clone(),
node_count,
Epoch::new(END_EPOCH).start_slot(slots_per_epoch),
slot_duration,
),
);
```

# Ethereum Foundation 테스트 벡터

## 1. EF Tests ( `ef_tests/` )

Ethereum Foundation의 공식 테스트 벡터를 실행하는 프레임워크입니다.

### 테스트 벡터 다운로드 및 실행

```
# 테스트 벡터 다운로드 (수백 MB)
make

# 테스트 실행
cargo test --features ef_tests

# 디버깅을 위한 병렬화 비활성화
cargo test --features ef_tests,disable_rayon -- --nocapture
```

### 주요 테스트 카테고리

```
// testing/ef_tests/src/cases.rs
mod bls_aggregate_sigs; // BLS 서명 집계
mod bls_aggregate_verify; // BLS 집계 검증
mod bls_batch_verify; // BLS 배치 검증
mod epoch_processing; // 에포크 처리
mod fork_choice; // 포크 선택
mod genesis_initialization; // 제네시스 초기화
mod kzg_blob_to_kzg_commitment; // KZG 커밋먼트
mod operations; // 블록 연산
mod rewards; // 보상 계산
mod sanity_blocks; // 블록 정상성 검사
mod sanity_slots; // 슬롯 정상성 검사
```

```
mod shuffling; // 검증자 셔플링

mod ssz_static; // SSZ 직렬화

mod transition; // 상태 전이
```

### 2. 상태 전이 테스트 벡터 ( `state_transition_vectors/` )

Lighthouse 특화 상태 전이 테스트 벡터를 생성하고 실행합니다.

**테스트 벡터 구조**

```
vectors/

└── exit/

├── invalid_bad_signature/

| ├── block.ssz # 테스트 블록

| ├── error.txt # 예상 에러

| └── pre.ssz # 사전 상태

├── valid_genesis_epoch/

| ├── block.ssz # 테스트 블록

| ├── post.ssz # 사후 상태

| └── pre.ssz # 사전 상태

└── ...
```

## 로컬 테스트넷 구성

### 1. Kurtosis 기반 테스트넷 ( `scripts/local_testnet/` )

Kurtosis를 사용하여 완전한 로컬 테스트넷을 구성할 수 있습니다.

### 설치 요구사항

```
# Docker 설치
sudo docker run hello-world

# Kurtosis 설치
kurtosis version

# yq 설치 (Ubuntu)
snap install yq
```

### 테스트넷 시작

```
cd ./scripts/local_testnet
./start_local_testnet.sh
```

### 주요 스크립트 옵션

```
# 도움말 보기
./start_local_testnet.sh --help

# 옵션들:
# -e: enclave 이름 (기본값: local-testnet)
```

```
# -b: Lighthouse 도커 이미지 빌드 여부 (기본값: true)
# -n: 네트워크 매개변수 파일 경로
# -p: 빌더 제안 활성화
# -c: CI 모드 (Grafana, Dora 탐색기 없이 실행)
# -a: Assertoor 테스트 실행
# -k: 기존 enclave 유지

# 예시: 빌드 없이 시작
./start_local_testnet.sh  -b  false
```

### 네트워크 매개변수 설정

```yaml
# scripts/local_testnet/network_params.yaml
participants:
- cl_type: lighthouse
cl_image: sigp/lighthouse:latest
el_type: geth
el_image: ethereum/client-go:latest
count: 4

network_params:
network_id: "3151908"
deposit_contract_address: "0x4242424242424242424242424242424242424242"
seconds_per_slot: 12
```

```
slots_per_epoch: 32
genesis_delay: 120
```

## 2. 테스트넷 모니터링 및 로그

### 서비스 상태 확인

```
# 실행 중인 서비스 확인
kurtosis enclave inspect local-testnet

# 로그 확인
kurtosis service logs local-testnet cl-1-lighthouse-geth
kurtosis service logs local-testnet vc-1-geth-lighthouse
kurtosis service logs local-testnet el-1-geth-lighthouse

# 모든 로그 덤프
kurtosis dump output_directory
```

### Dora 탐색기 접근

```
# Dora 탐색기 열기
open $(kurtosis port print local-testnet dora http)

# Kurtosis 웹 UI
```

```
kurtosis web
```

**테스트넷 중지**

```
cd ./scripts/local_testnet

./stop_local_testnet.sh
```

# 특화 테스트 도구들

## 1. 실행 엔진 통합 테스트 ( `execution_engine_integration/` )

실행 레이어와의 통합을 테스트하는 도구입니다.

```
// 실행 엔진 API 테스트

// - Engine API 호출 검증

// - 페이로드 빌딩 테스트

// - 포크 선택 업데이트 검증
```

## 2. Web3Signer 테스트 ( `web3signer_tests/` )

외부 서명 서비스와의 통합을 테스트합니다.

```
// Web3Signer 통합 테스트

// - 원격 서명 검증

// - TLS 연결 테스트
```

```
// - 슬래싱 보호 검증
```

### 3. 네트워크 테스트 (`network_testing/`)

P2P 네트워킹 기능을 테스트하는 도구들입니다.

```
// 네트워크 테스트 도구
// - 피어 연결 테스트
// - Gossipsub 메시지 전파 검증
// - 디스커버리 프로토콜 테스트
```

## 테스트 실행 가이드

### 1. 단위 테스트 실행

```
# 전체 테스트 스위트 실행
cargo test

# 특정 모듈 테스트
cargo test --package beacon_node

# 병렬 실행 제한 (메모리 부족 시)
cargo test -- --test-threads=1
```

### 2. 통합 테스트 실행

```
# EF 테스트 실행
cd testing/ef_tests
make
cargo test --features ef_tests

# 시뮬레이션 실행
cd testing/simulator
cargo run -- basic-sim --nodes 4 --validators-per-node 8

# 상태 전이 벡터 테스트
cd testing/state_transition_vectors
make test
```

### 3. 로컬 테스트넷 실행

```
# 기본 설정으로 테스트넷 시작
cd scripts/local_testnet
./start_local_testnet.sh

# 사용자 정의 설정
./start_local_testnet.sh -n custom_params.yaml -b false
```

## 성능 및 부하 테스트

### 1. 벤치마크 테스트

```
# Criterion 벤치마크 실행
cargo bench

# 특정 벤치마크
cargo bench --package state_processing
```

### 2. 메모리 프로파일링

```
# Valgrind를 사용한 메모리 검사
valgrind --tool=memcheck cargo test

# 힙 프로파일링
cargo test --features heap-profiling
```

## 결론

Lighthouse의 테스트 프레임워크는 다음과 같은 특징을 가집니다:

1. **포괄적 커버리지**: 단위 테스트부터 전체 네트워크 시뮬레이션까지
2. **프로덕션 동등성**: 실제 프로덕션 코드와 동일한 타입 사용
3. **자동화된 검증**: 다양한 네트워크 속성 자동 검증

4. **확장 가능성**: 새로운 테스트 시나리오 쉽게 추가 가능

5. **실용적 도구**: 개발자가 실제로 사용할 수 있는 로컬 테스트넷

이러한 테스트 인프라를 통해 Lighthouse는 높은 품질과 안정성을 유지하며, 복잡한 분산 시스템의 정확성을 보장합니다.